4. Exercise

Anomaly Detection Challenge

Deadline: **03.02.26**, Midnight

Wintersemester 2025/26

Patrick Schäfer, patrick.schaefer@hu-berlin.de

Agenda

- Questions?
- 4. Exercise
- Lösung 3. Bonusblatt

Question?

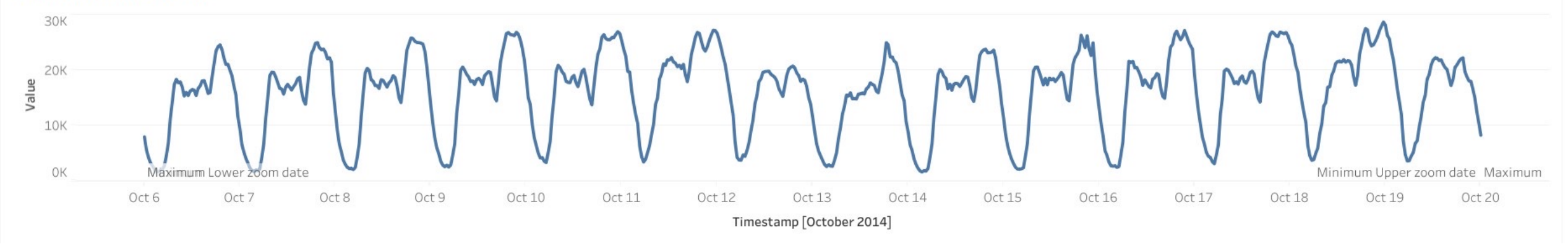
Agenda

- Questions?
- 4. Exercise
  - **Data**
  - Part 1
  - Part 2
  - Part 3

Time Series

- A time series is a sequence of real values that is recorded over time

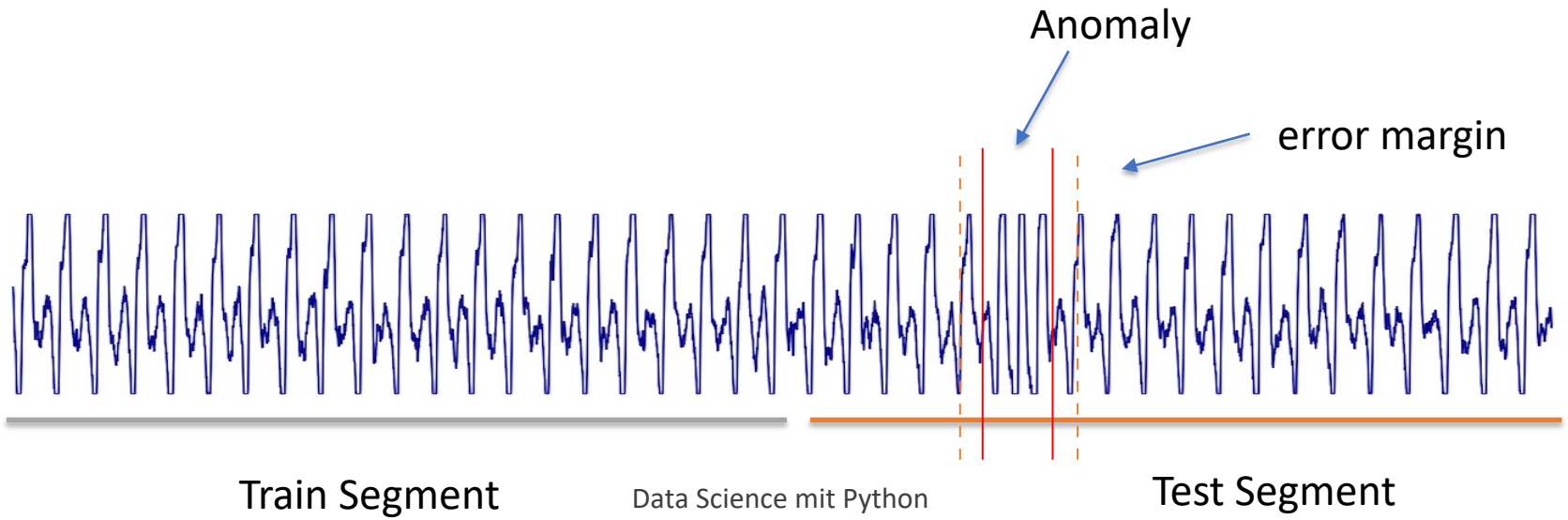
$$X = (x_1, \ldots, x_n):$$



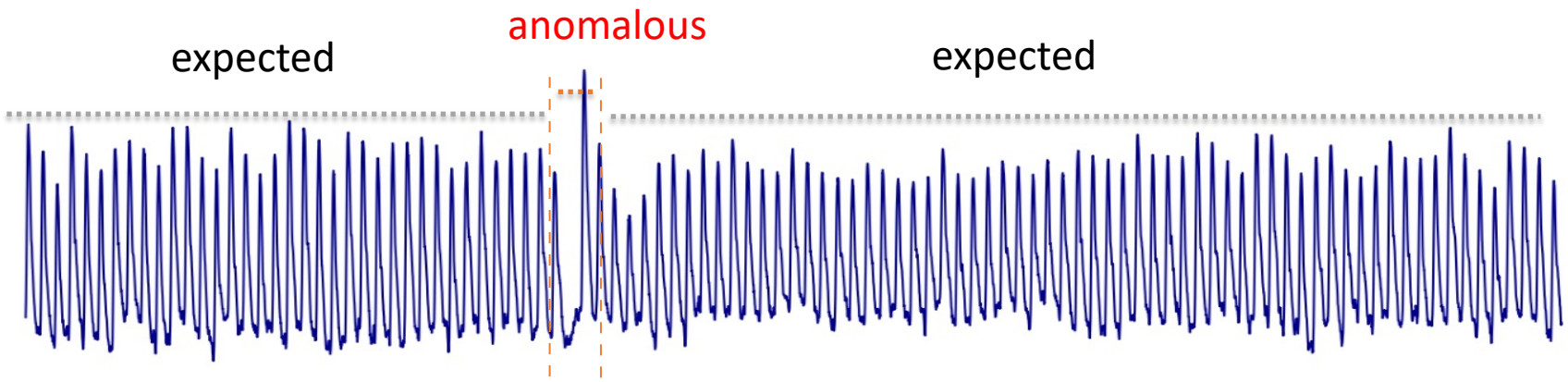
Weekly taxi passenger traffic in NY

Time Series Anomaly Detection

- You are given 30 time series
  - Each containing exactly **one anomaly of unknown size and shape**
  - The train segment (prefix) has no anomaly
  - The anomaly is contained in the **test segment** (suffix)
  - **You are not given the anomalies**

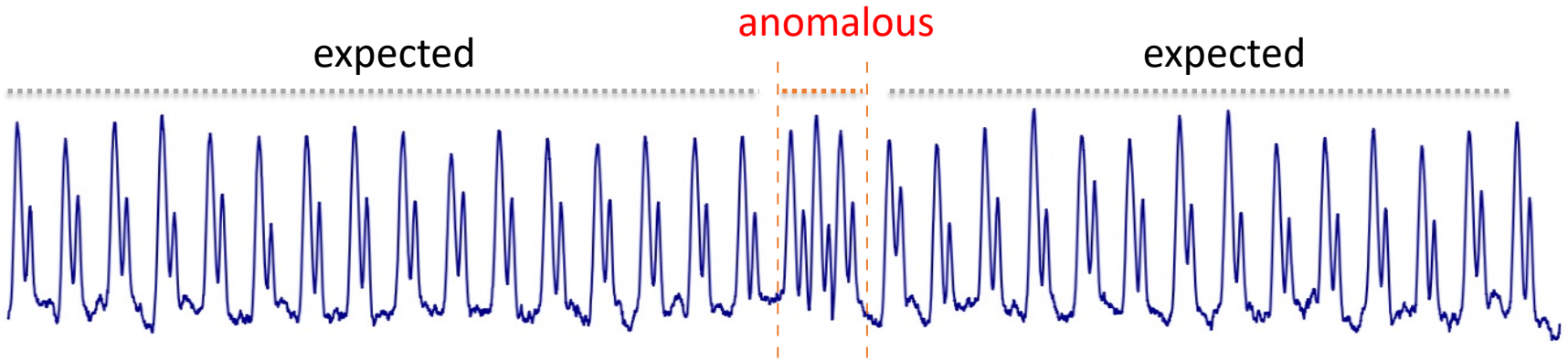


Types of Anomalies - Point Anomalies



Point Anomalies are data points that deviate in deflection

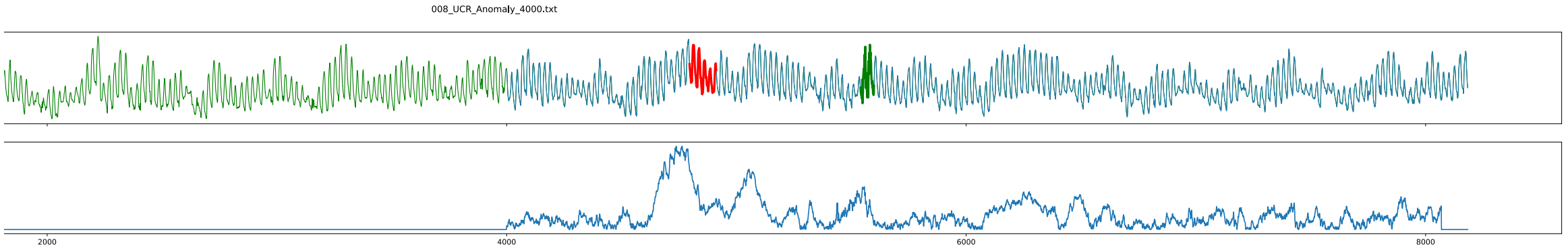
Types of Anomalies - Subsequence Anomalies



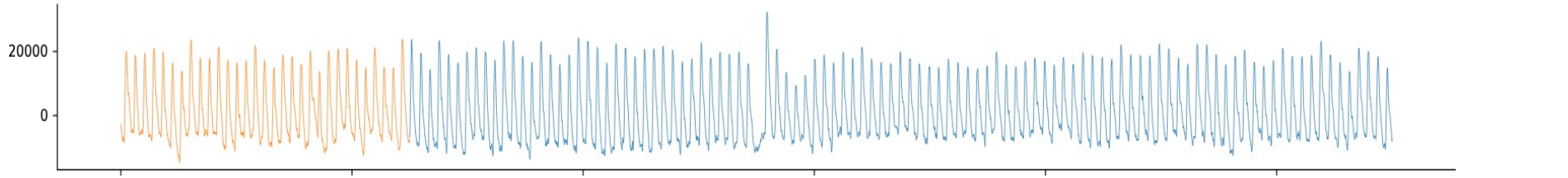
- **Subsequence Anomalies** are temporal patterns that deviate in shape

Lessons to be Learned

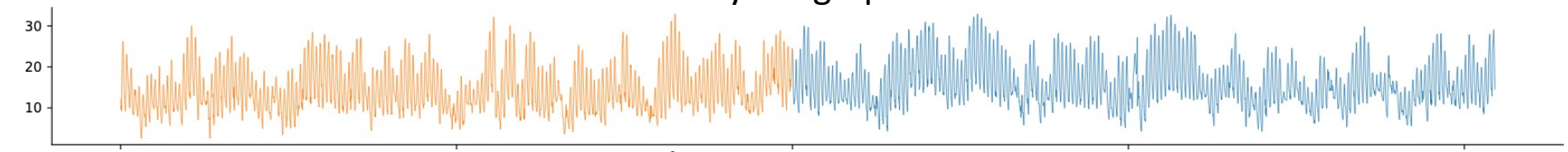
- Largest improvements result from **inspecting and pre-processing** (addressing) the errors your model makes
- This is what you will practice in this exercise



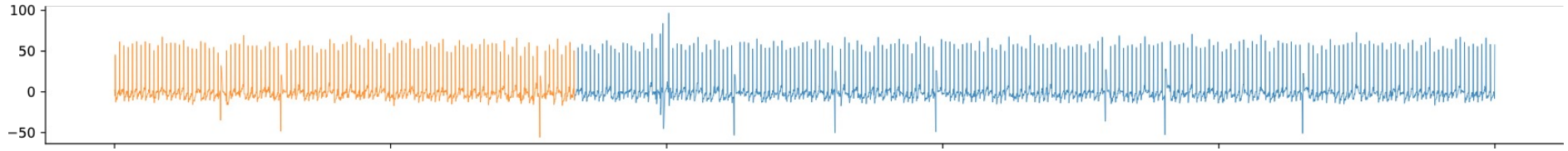
Predicted (Red) vs True (Green) Anomaly



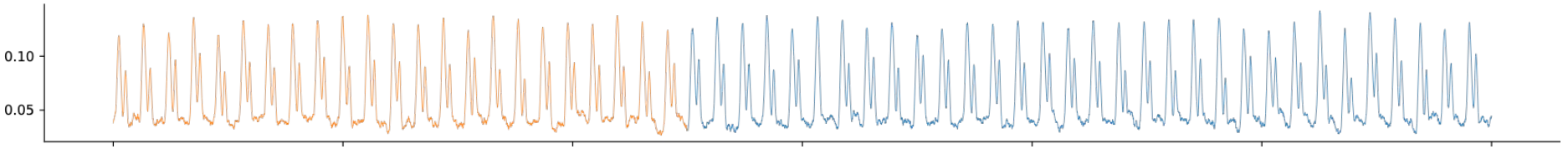
Plethysmograph



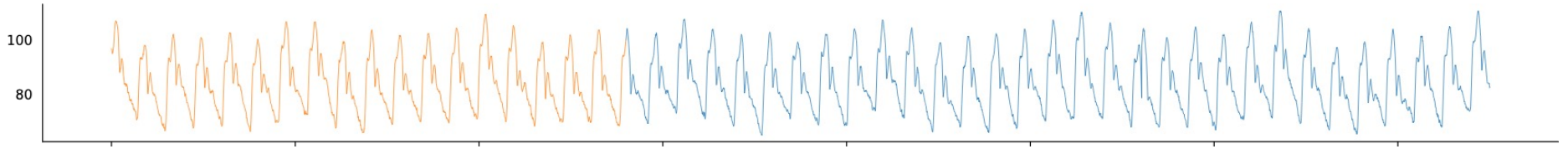
Air Temperature



ECG



Gait

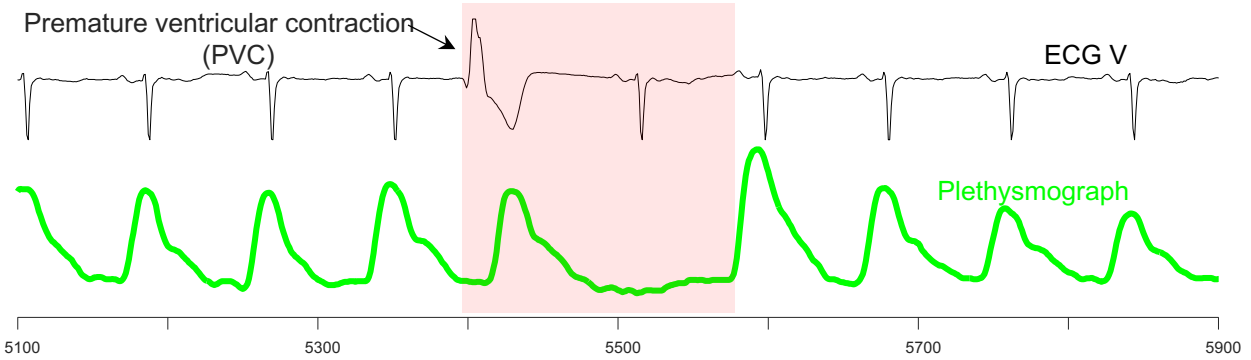


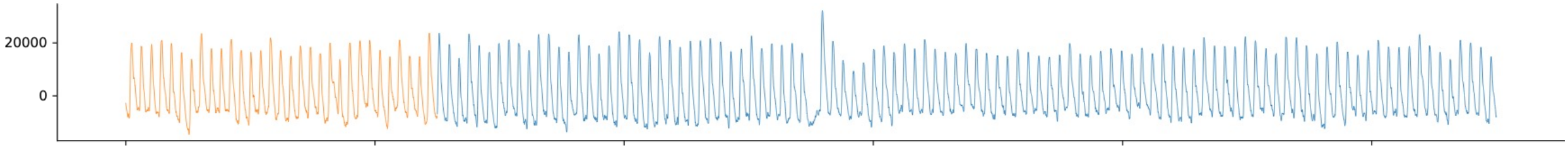
Blood Pressure

30 time series derived from 6 use cases

DS 1 - Plethysmograph Signals - BIDMC

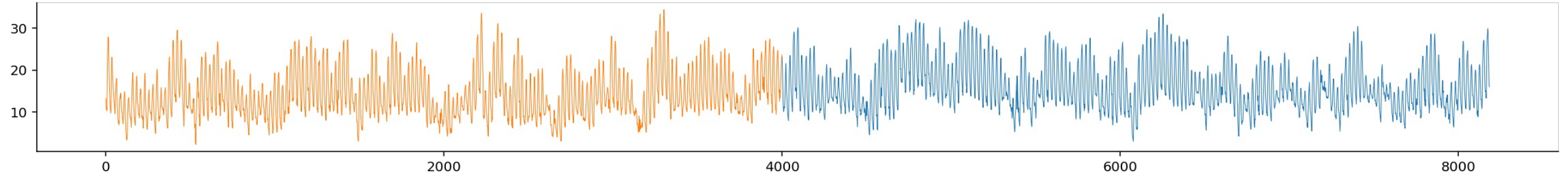
- A plethysmograph is an instrument for measuring changes in volume within an organ or whole body
- The recording has an abnormal heartbeat, a PVC





DS 2 - Air Temperature

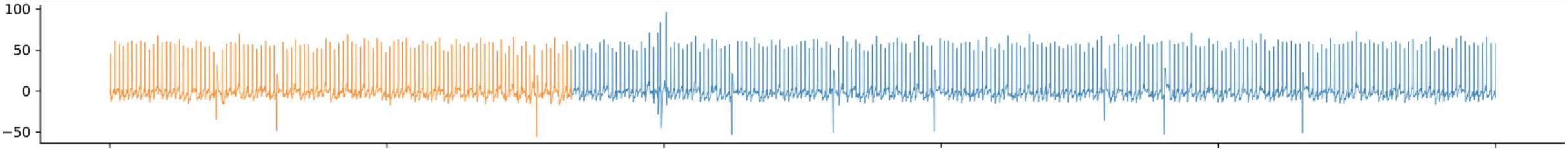
- The data is from a public weather data report from CIMIS station 44 in Riverside [1]
- Dataset consists of hourly air temperature between 03/01 and 03/31 from 2009 to 2019
- The anomaly is synthetic



[1] ftp://ftpcimis.water.ca.gov/pub2/annualMetric/

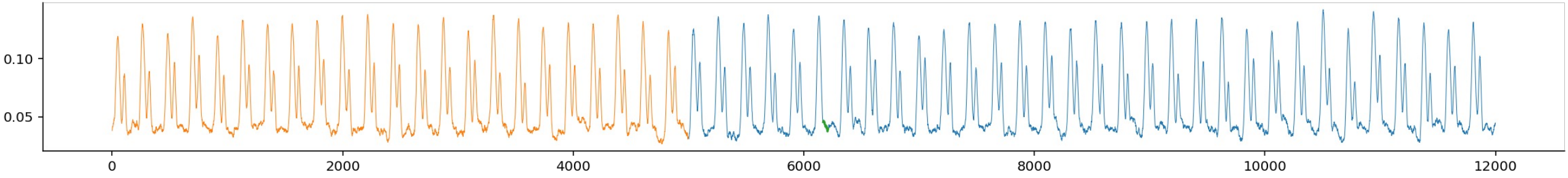
DS 3 - ECG

- No further information available



DS 4 - Gait Phase

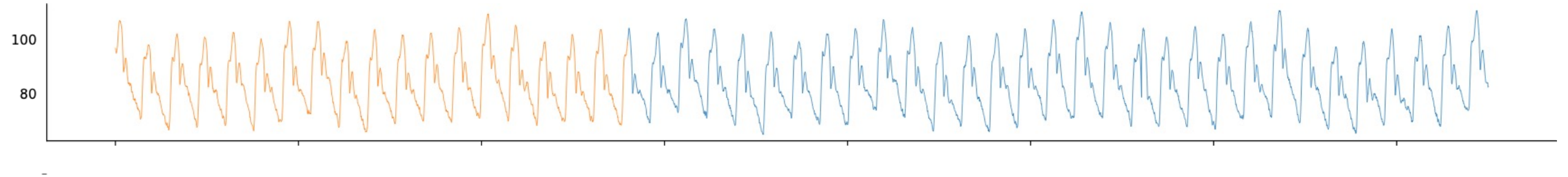
- The data comes from subject 7 in the GaitPhase Database [1]
- The subject was required to walk on a split-belt treadmill at walking speed of 1.1 m/s.
- This dataset represents the vertical direction of a 3D marker's position
- The marker was placed on subject's left shoe
- There are totally 55 gait cycles in the dataset
- The anomaly is synthetic



[1] https://www.mad.tf.fau.de/research/activitynet/gaitphase-database/

DS 5 - Internal Bleeding

- The data comes from an internal bleeding dataset[1]
- From the dataset, we extracted the arterial blood pressure measurements of pigs.
- The anomaly is synthetic



[1]: http://mathieu.guillame-bert.com

4. Exercise

- The exercise consists of **3 parts**

Part 1) Exploratory Data Analysis

Part 2) Vibe Coding

Part 3) Anomaly Detection Challenge

Agenda

- Questions?
- 4. Exercise
  - **Part 1 – EDA**
  - Part 2 – Vibe Coding
  - Part 3 – Challenge

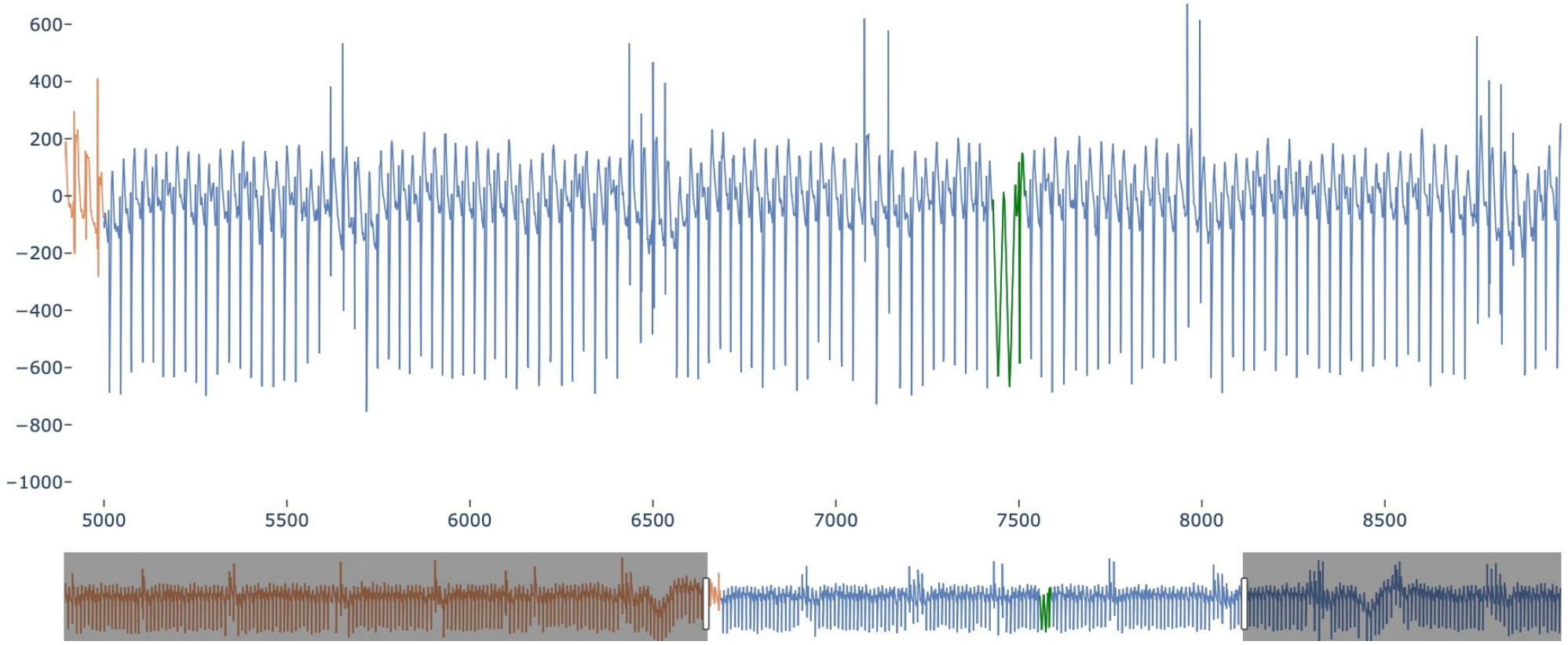
Part 1 - Exploratory Anomaly Detection

- You are not given the anomalies
  - You know, there is one in the test segment (suffix) for each time series
  - You must recover them from the time series yourself
  - Must manually identify **10 out of 30** anomalies **correctly**

- **Task: Identify Anomalies by Inspection**

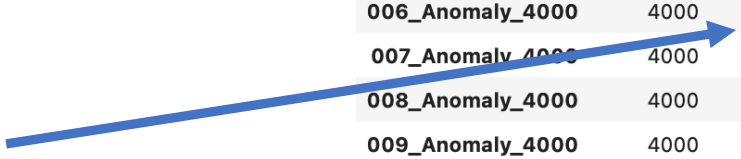
Identify Anomalies by Visual Inspection

- You are given a ’plotly’-primer to zoom into the time series
  - primer_visualization.ipynb



Task

- Manually identify the anomalies and update the labels.csv file
- These annotations will help you in Task 3, too



Name	TrainSplit	Start	End
001_Anomaly_5000	5000	7428	7517
002_Anomaly_4375	4375	-1	-1
003_Anomaly_4375	4375	-1	-1
004_Anomaly_2500	2500	-1	-1
005_Anomaly_4000	4000	-1	-1
006_Anomaly_4000	4000	-1	-1
007_Anomaly_4000	4000	-1	-1
008_Anomaly_4000	4000	-1	-1
009_Anomaly_4000	4000	-1	-1
010_Anomaly_4000	4000	-1	-1
011_Anomaly_3333	3333	-1	-1
012_Anomaly_5000	5000	-1	-1
013_Anomaly_5000	5000	-1	-1
014_Anomaly_2666	2666	-1	-1
015_Anomaly_250	250	-1	-1
016_Anomaly_1666	1666	-1	-1
017_Anomaly_1666	1666	-1	-1
018_Anomaly_2666	2666	-1	-1
019_Anomaly_5000	5000	-1	-1
020_Anomaly_5000	5000	-1	-1
021_Anomaly_5000	5000	-1	-1
022_Anomaly_4000	4000	-1	-1
023_Anomaly_5000	5000	-1	-1
024_Anomaly_3200	3200	-1	-1
025_Anomaly_2800	2800	-1	-1
026_Anomaly_1700	1700	-1	-1
027_Anomaly_1200	1200	-1	-1
028_Anomaly_1600	1600	-1	-1
029_Anomaly_2300	2300	-1	-1
030_Anomaly_3000	3000	-1	-1

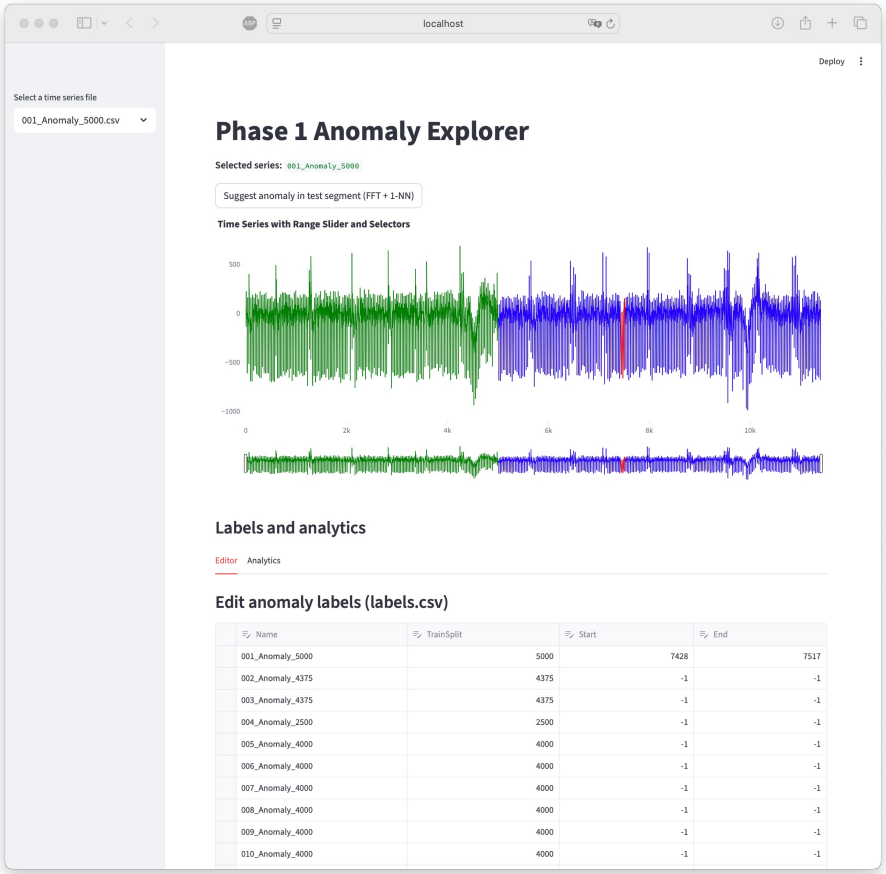
Agenda

- Questions?
- 4. Exercise
  - Part 1 – EDA
  - **Part 2 – Vibe Coding**
  - Part 3 – Challenge

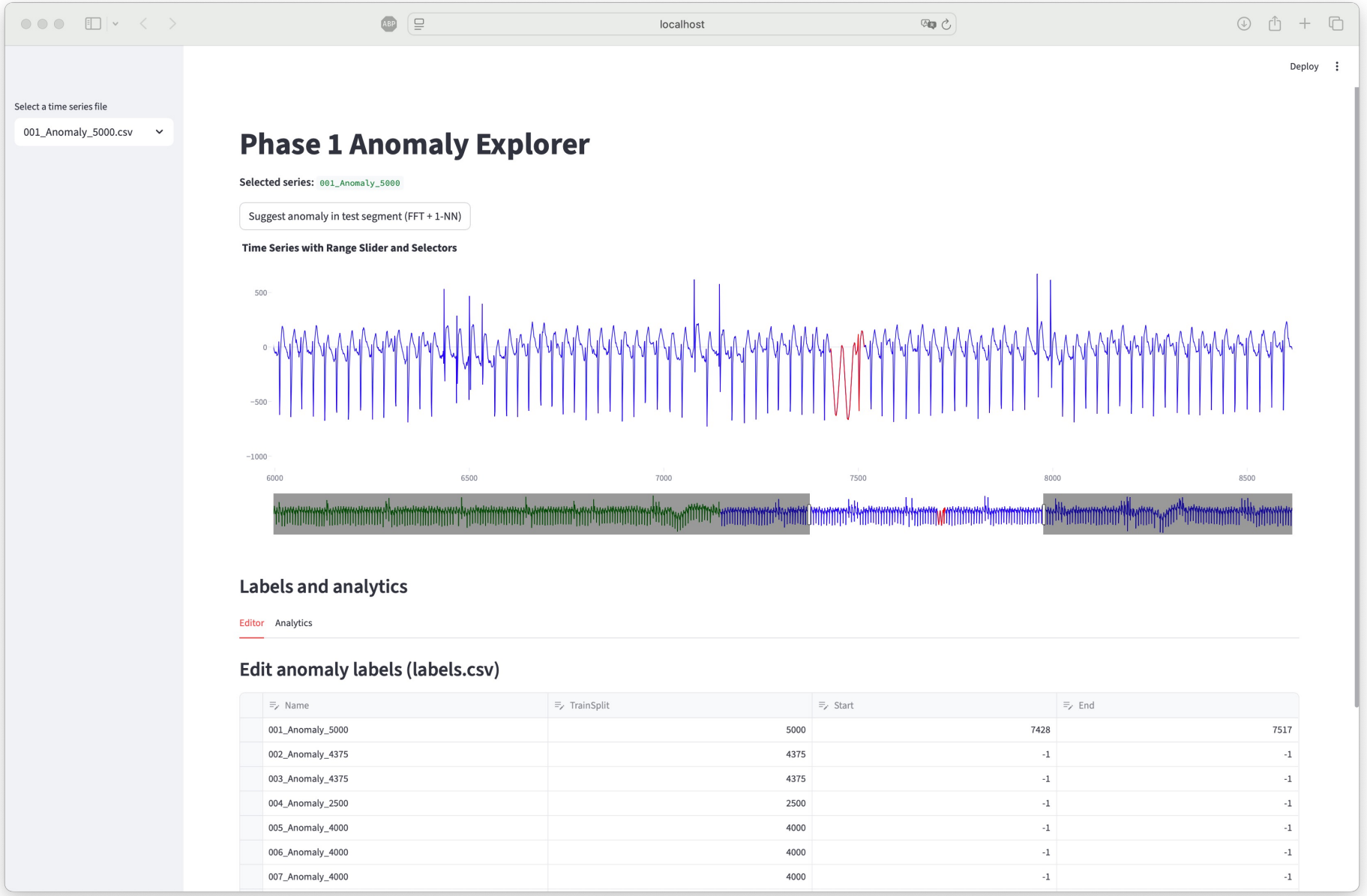
Part 2 – Vibe Coding Task

- Build a Streamlit app
  - You can **directly use the code from part 1 in the prompt**

- **Required features:**
  - Allow the user to select a time series.
  - Display the time series (train and test) together with the annotated anomaly.
  - Provide interactive zooming into the time series.
  - Allow the user to modify the anomaly annotation and update the display accordingly.
  - Provide an option to export the labels.



Example



- You can use the streamlit app to identify the anomalies from Part 1, too

Agenda

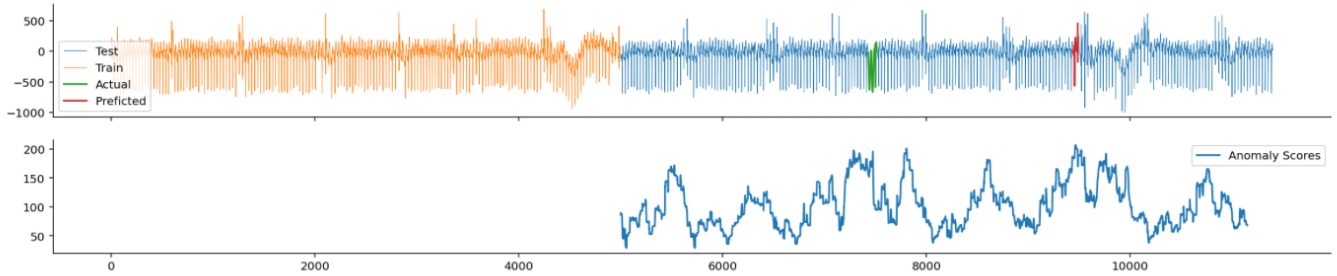
- Questions?
- 4. Exercise
  - Part 1 – EDA
  - Part 2 – Vibe Coding
  - **Part 3 – Challenge**

Part 3: Anomaly Detection Challenge

- You are given a primer for implementing anomaly detectors
  - primer_detector.ipynb

Anomaly Detection using Predictive Modelling

You are given wrappers, which you can use to implement methods. Each wrapper returns a score-profile and the predicted anomaly. The score's maximum indicates where the anomaly is.




```python
# Distance-based anomaly scores
# e.g. k-Nearest Neighbor Classifier
# see: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html
def nearest_neighbor_based_score(X, test_start):
    score = np.zeros(len(X))
    # TODO: fill in code here as needed

    predicted = test_start + np.argmax(score[test_start:])
    return score, predicted
```

Primer

- When running the code, it shows each time series along the predicted anomaly and the scores
- If you annotated anomalies, these will also be shown

Predicted Anomaly

Anomaly Scores

Task

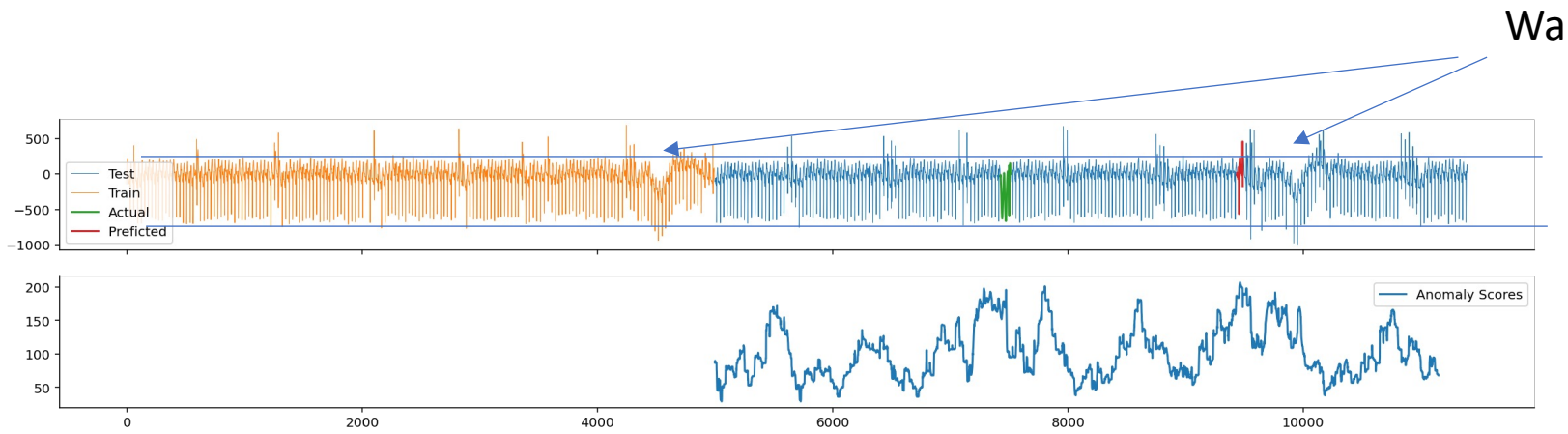
You may use Vibe Coding!

- Your task is to implement at **least three different approaches**
- **Two must come from different categories**
  1. Distance-Based
     (e.g., k-Nearest Neighbour Classifier)
  2. Model-Based
     (e.g., Isolation Forest, LOF, One-Class-SVM)
  3. Regression-Based
     (e.g., linear / polynomial regression)
  4. Forecasting-Based
     (e.g., Prophet, ARIMA)
  5. Clustering-Based
     (e.g., DBSCAN)
  6. Statistics-Based
     (e.g., IQR, KDE)
- A third approach can be a **different kind of pre-processing**

```python
# Distance-based anomaly scores
# e.g. k-Nearest Neighbor Classifier
# see: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html
def nearest_neighbor_based_score(X, test_start):
    score = np.zeros(len(X))
    # TODO: fill in code here as needed

    predicted = test_start + np.argmax(score[test_start:])
    return score, predicted


# Use a classification approach to detect anaomalies
# e.g. e.g., Isolation Forest, LOF, One-Class-SVM
# see: https://scikit-learn.org/stable/modules/outlier_detection.html
def classifcation_based_scores(X, test_start):
    score = np.zeros(len(X))
    # TODO: fill in code here as needed

    predicted = test_start + np.argmax(score[test_start:])
    return score, predicted



# Use a regression-based approach to detect anaomalies
# e.g. linear / polynomial regression
# see: https://scikit-learn.org/stable/modules/linear_model.html
def regression_based_scores(X, test_start):
    score = np.zeros(len(X))
    # TODO: fill in code here as needed

    predicted = test_start + np.argmax(score[test_start:])
    return score, predicted


# Use a forecasting-based approach to detect anaomalies
# e.g. ARIMA, Prophet
# see: https://medium.com/@aeon.toolkit/blazingly-fast-arima-with-aeon-9796d7015cdd
def forecasting_based_scores(X, test_start):
    score = np.zeros(len(X))
    # TODO: fill in code here as needed

    predicted = test_start + np.argmax(score[test_start:])
    return score, predicted
```

Role of Pre-Processing

- There is no recipe for finding good solutions => **trial and error**
- Start by looking at the **mistakes your model** to get ideas how to handle these



- The time series is wandering around the mean => maybe we can remove the trend?
- The data has some extreme values => maybe we can floor or cap the data

Preprocessing – Examples

- Zero-Mean-Normalization

```
# zero-mean-normalization
data = (data - np.mean(data)) / np.std(data)
```

- Remove trends by Rolling Means on Time Series

```
# remove trend by moving averages
means = pd.Series(data).rolling(ws, min_periods=1).mean().to_numpy()
data = (data[:len(data)-int(ws/2)] - means[int(ws/2):])
```

Preprocessing – Examples

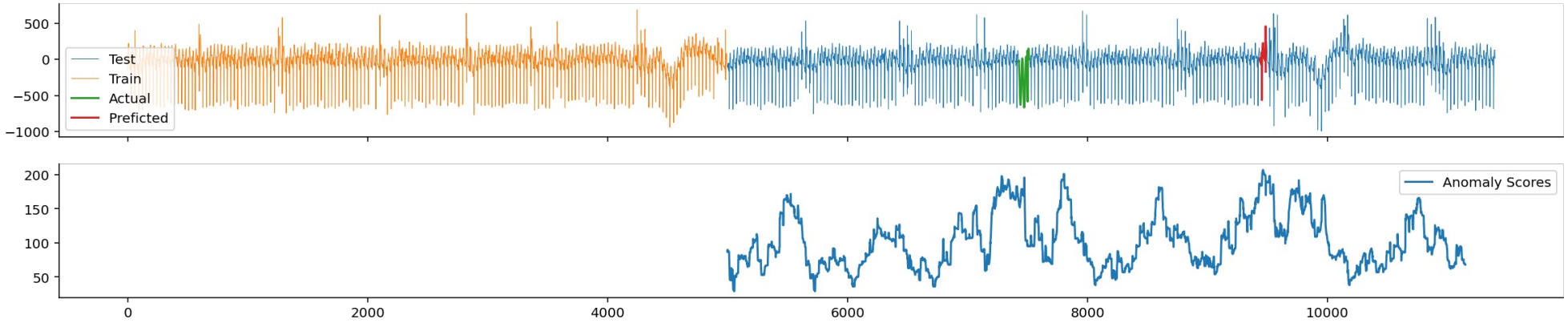
- Flooring and Capping the time series

```
# outlier removal
lower_perc = np.percentile(data, 10)
upper_perc = np.percentile(data, 92)
data[data > upper_perc] = upper_perc
data[data < lower_perc] = lower_perc
```

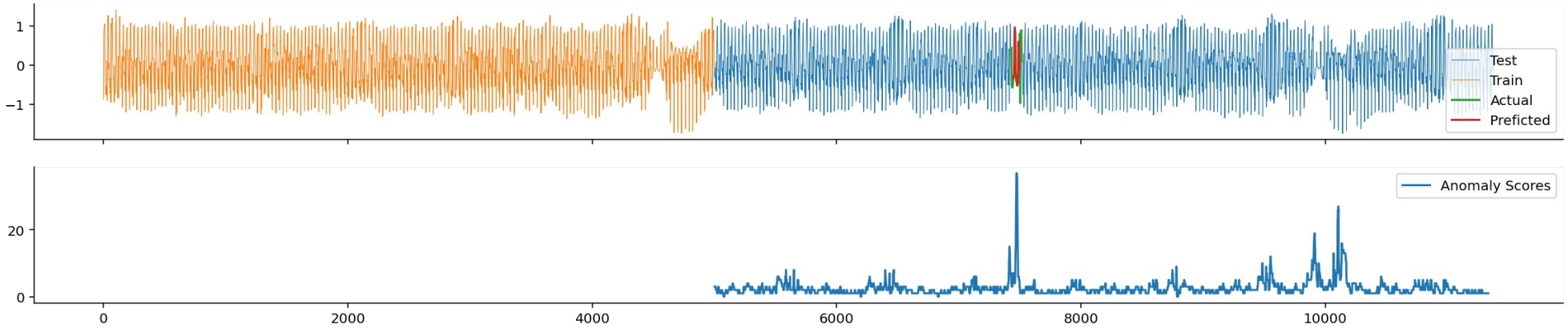
- Compare the peak in the training data with the peak in test data

```
# Apply lowpass first, if the anomaly-peak is found in the train-data
if np.max(profile[:test_start])*1.1 > np.max(profile[test_start:]):
    data = pd.Series(data).rolling(7, min_periods=1).min().to_numpy()
    profile = compute_distances(data, ws, k)
```

- **Prior** to preprocessing



- **After** preprocessing



Some Suggestions I/II

- Standardization or Normalization
- Hyperparameter Tuning
  - Try ranges of window sizes ws (i.e. multiples of dominant frequency)
  - Floor/cap the window-size to a minimum and maximum value
- Different window-size selection
  - Based on autocorrelation
  - Based on some fancy Paper
- Apply Smoothening

Some Suggestions II/II

- Log Transform
- Flooring or Capping of extreme values
- Sampling (use every x-th value)
- Combinations of the above
- A different peak finder (not using the maximum)?

- ... your own idea?
- Ask ChatGPT?

Preprocessing and Feature Engineering

- It is an **art** that involves a lot of trial and error and experience
- There is no recipe to what must work

- **Inspect the data and the mistakes** your model makes
  - Apply exploratory analysis
  - Learn from the mistakes your model makes and find ways to address these

- **It is fine, if you cannot improve the model**
  - It is about testing ideas

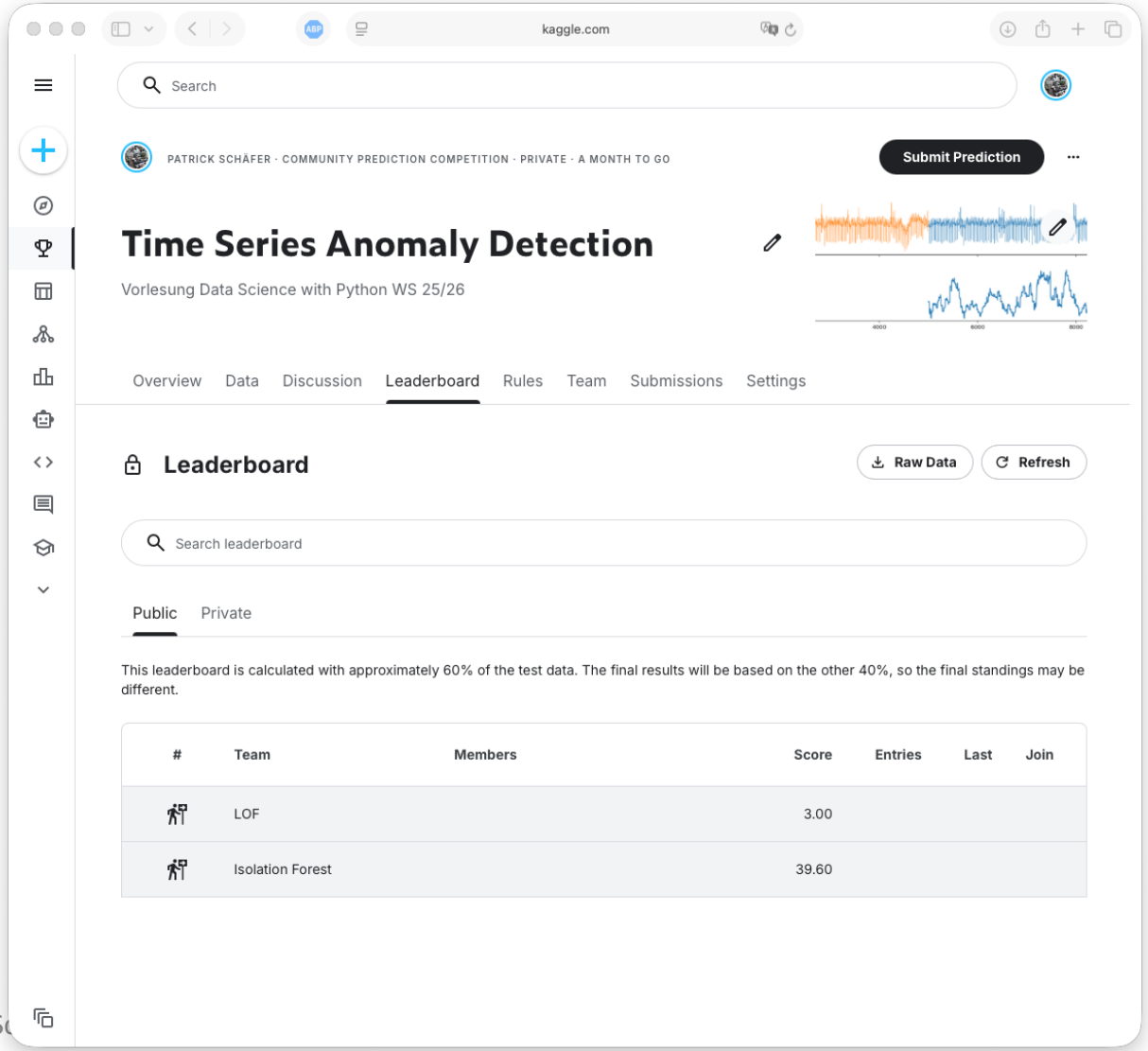
Kaggle

- We will be using Kaggle for managing submissions
- You may submit **three predictions** every day until the end
- **And you must make three submissions!**

- When submitting predictions, you will see your score on the leader board

Leaderboard

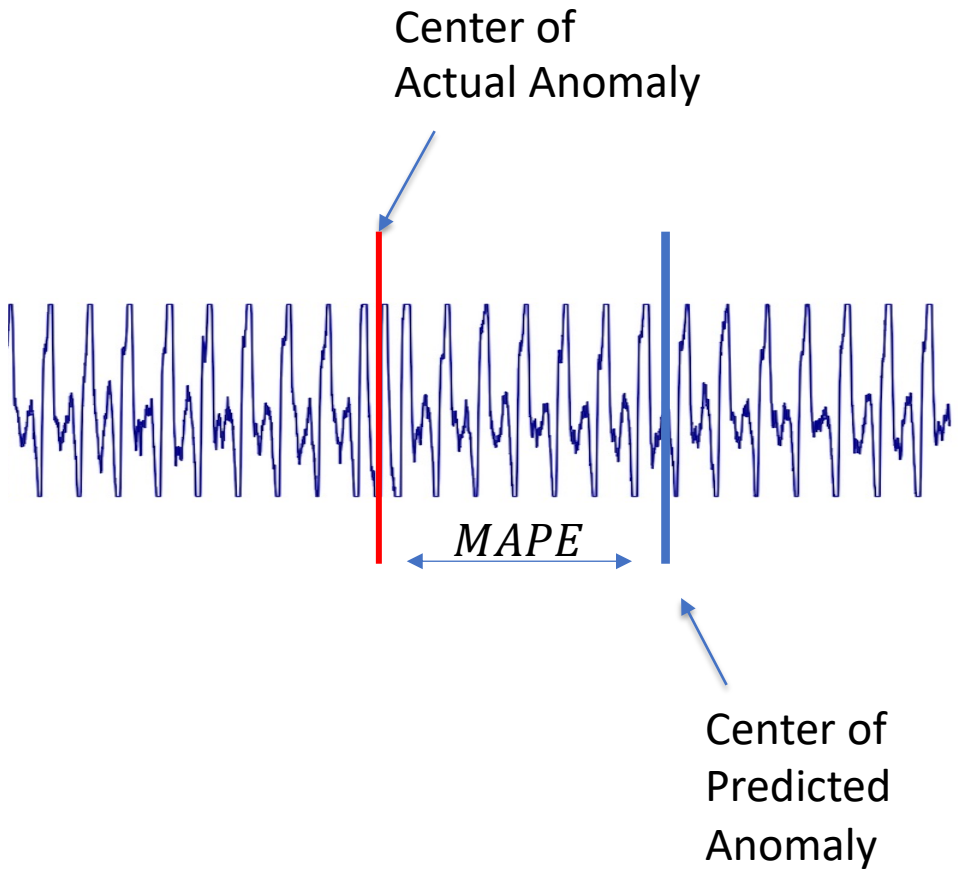
- The public leader board is based on **60% of the data**
  - You do not know, which samples
- The final standings are based on the **remaining 40% of the data**
  - You do not know, which samples
  - Revealed at the end of the competition
- There are **two baselines given**:
  - Isolation Forests
  - One-Class SVM



Used Metric: MAPE

- We will be using the Mean Absolute Percentage Deviation (MAPE)
- $MAPE = \frac{1}{n}\sum_{i=1}^{n}\frac{|A_i - F_i|}{A_i}$
  - $A_i = Actual\ Anomaly$
  - $F_i = Predicted\ Anomaly$
  - n = number of time series, i.e. 30 or 150

- **The further away your prediction is from the anomaly, the higher the error is**



Kaggle – URL to the competition

- **Available Start of January!**
  - Phase 1 View:
    - https://www.kaggle.com/competitions/time-series-anomaly-detection-exercise-WS-25-26
  - Phase 1 Register:
    - https://www.kaggle.com/t/35e269015d834d06b6017d62e6650875

Super-Human Performance Levels

- Did the algorithm help you improve your annotated anomalies?
- Was the algorithm better in spotting anomalies than you?

Submission Deadline: 03.02.26

- The exercise is considered to have been **successfully solved,** if
  - Part 1: You correctly identified **10 (out of 30) anomalies**
  - Part 2: Your app **can be executed** and contains **the requested features**
  - Part 3: You made at **least three submissions to Kaggle with three different approaches**
- **Deliverables:**
  - **Part 1: EDA (1 File):** The CSV with your annotations
  - **Part 2: Vibe Coding:**
    - **The file** with **python code**
    - **One file** documenting the **used prompts** and the **used LLM**
    - **Screenshot(s)** or a small video **of your app**
  - **Part 3: Challenge** (**6-9 Files**):
    a) **Executed** Notebook and HTML export of **three solutions**
    b) **Kaggle Submissions in CSV Format**

Bonusblatt

Agenda

- Questions?
- 4. Exercise
- **Lösung 3. Bonusblatt**

Aufgabe 1 (Verteilungen)

Wir haben für $m = 5$ Besucher erfasst, wie häufig sie im Monat ins Kino gehen:

ID	Alter	Besuche
1	32	5
2	8	1
3	30	4
4	14	2
5	26	3

Tabelle 1: Monatliche Besuche eines Kinos.

a) Berechnen Sie anhand von Tabelle 1 nachfolgende Statistiken:

1) Durchschnittliches Alter $\bar{X} = mean(Alter)$:

2) Durchschnittliche Besuche $\bar{Y} = mean(Besuche)$:

3) Berechnen Sie die Einträge der Kovarianzmatrix Σ für die Spalten *Alter* und *Besuche*.
Hinweis: verwenden Sie die Kovarianz-Formel mit Bessel-Korrektur:

$$COV(X, Y) = \frac{\sum_{i=1}^{m}(X^{(i)} - \bar{X})(Y^{(i)} - \bar{Y})}{m - 1}$$

A) $COV(Alter, Alter)$

B) $COV(Besuche, Besuche)$

C) $COV(Alter, Besuche)$

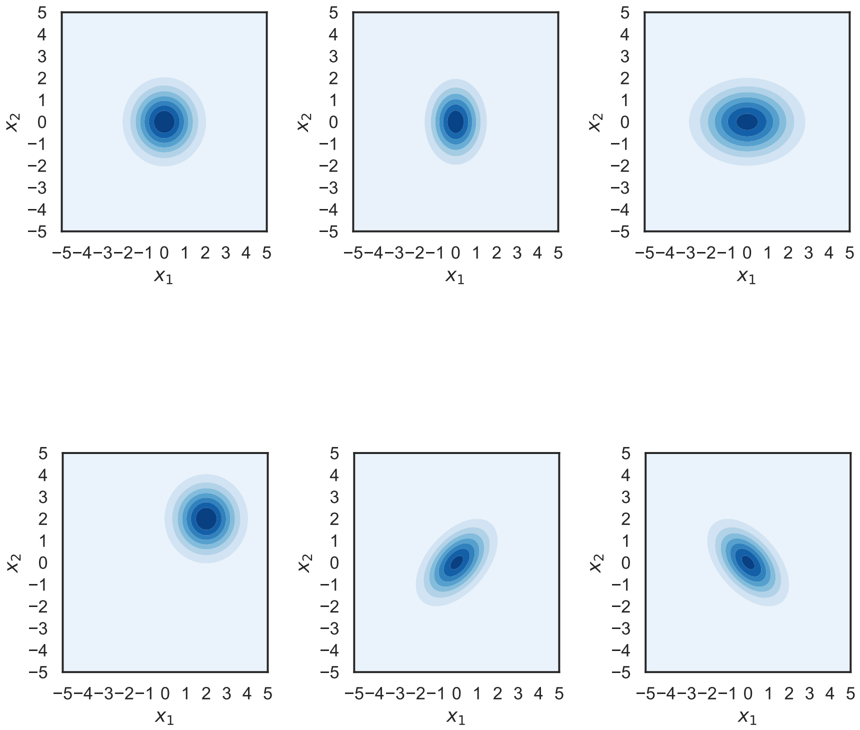
b) Was sagt die Kovarianz $COV(Alter, Besuche)$ von Alter und Besuche über die Verteilung dieser Daten aus? Und wie könnten Sie dies visualisieren?

c) Über Mittelwert $\mu = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \end{bmatrix}$ und Kovarianzmatrix $\Sigma = \begin{bmatrix} VAR(x_1) & COV(x_1, x_2) \\ COV(x_2, x_1) & VAR(x_2) \end{bmatrix}$ der 2-dim Gaußverteilung $N(x, \mu, \Sigma)$ können wir die Ausrichtung der Dichteverteilung bestimmen. Gegeben sind die sechs Paare:

(a) $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \sum = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$ (b) $\mu = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \sum = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (c) $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \sum = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}$

(d) $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \sum = \begin{bmatrix} 0.6 & 0 \\ 0 & 1 \end{bmatrix}$ (e) $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \sum = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (f) $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \sum = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$

Beschriften Sie die nachfolgenden Contour-Plots der Gauß-Verteilung mit den passenden Parametern. Schreiben Sie je ein Symbol aus (a) bis (f) unter dem jeweiligen Plot:



Aufgabe 2 (Gaußsche Mischverteilungen)

Ein Gaußsches Mischmodell enthält $k = 3$ Verteilungen, die anhand ein-dimensionaler Trainingsdaten berechnet wurden. Die drei gaußschen Verteilungen

$$P_i(x) = \frac{1}{\sigma_i \cdot \sqrt{2\pi}} \cdot e^{-\frac{1}{2} \cdot \left(\frac{x-\mu_i}{\sigma_i}\right)^2}$$

sind dabei gegeben durch:

$$\begin{array}{ccc} \mu_1 = -1 & \mu_2 = 1 & \mu_3 = 4 \\ \sigma_1 = 1 & \sigma_2 = 2 & \sigma_3 = 0.4 \end{array}$$

Für diese Verteilungen sind nachfolgend für Sie die Werte an verschiedenen Stützwerten x_i vorab berechnet worden:

x	-4	-3	-2	-1	0	1	2	3	4	5	6
$P_1(x)$	0.001	0.023	0.24	0.4	0.24	0.05	0	0	0	0	0
$P_2(x)$	0.006	0.023	0.06	0.12	0.18	0.2	0.18	0.12	0.06	0.03	0.01
$P_3(x)$	0	0	0	0	0	0	0	0.04	1	0.04	0

Zusätzlich sind nachfolgende absoluten Häufigkeiten der Klassen in den Trainingsdaten gegeben:

$$y = 1 : 40$$
$$y = 2 : 20$$
$$y = 3 : 40$$

a) Berechnen Sie die relativen Häufigkeiten $p(y = 1), p(y = 2), p(y = 3)$ anhand der Trainigsdaten.

2 b) Berechnen Sie für jeden der 6 Datenpunkte $x \in \{-2, 0, 2, 4, 5, 6\}$ eine Vorhersage $\hat{y}$ mittels Bayes Ansatz $p(x, y)$ analog zur Vorlesung.

	$p(x, y = 1)$	$p(x, y = 2)$	$p(x, y = 3)$	predicted $\hat{y}$
$x = -2$				
$x = 0$				
$x = 2$				
$x = 4$				
$x = 5$				
$x = 6$				

x	-4	-3	-2	-1	0	1	2	3	4	5	6
$P_1(x)$	0.001	0.023	0.24	0.4	0.24	0.05	0	0	0	0	0
$P_2(x)$	0.006	0.023	0.06	0.12	0.18	0.2	0.18	0.12	0.06	0.03	0.01
$P_3(x)$	0	0	0	0	0	0	0	0.04	1	0.04	0

Joint Probability

$$p(x, y) = p(x|y) \cdot p(y)$$

- $p(y = 1) = 0.4$
- $p(y = 2) = 0.2$
- $p(y = 3) = 0.4$

2 c) Was passiert, sobald wir aus den gegebenen Daten extrapolieren, d.h. in Regionen Vorhersagen treffen, in denen wir keine Trainingsdaten gemessern haben (z.B. $x >> 6$ oder $x << -2$)?